

Практическое задание: Реализация системы разделения ролей с базой данных

Задача

Реализовать серверную часть системы разделения ролей с использованием Node.js, Express и базы данных (через Sequelize).

Требования:

Базовый уровень без бд (оценка "3") – создать массив пользователей с ролями. Разграничить доступ к двум страницам в зависимости от роли.

Продвинутый уровень (оценка "4") – создать 2 таблицы в бд(пользователь и роль).

Разграничить доступ для пользователей.

Высокий уровень (оценка "5") – добавить к заданию регистрацию и авторизацию.

Пошаговое руководство для выполнения:

Шаг 1. Установка зависимостей

```
import express from 'express';
import session from 'express-session';
import bodyParser from 'body-parser';
import { Sequelize, DataTypes } from 'sequelize';
import path from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

Если ругается на require, то меняем его на импорт и для пакета path ставим доп пакет работы с путями(последние три строки.)

Здесь проблема в том что require используется при одной интерпретации кода, а import при другой. Меняться может из-за установки пакет или других действий. Обычно работаем в commonjs, а он включает es модули.

- В ES-модулях __dirname и __filename не определены.
- Вместо них используется import.meta.url, который предоставляет URL текущего файла. Чтобы преобразовать его в путь, используется fileURLToPath из модуля url.

Шаг 2. Настройка базы данных

```
const sequelize = new Sequelize('test1', 'admin', '1', {
  host: 'localhost',
  dialect: 'mysql'
});
```

1.

```
// Определение модели роли
const Role = sequelize.define('Role', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  name: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  }
});

// Определение модели пользователя
const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true,
  },
  login: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  roleId: {
    type: DataTypes.INTEGER,
    references: {
      model: Role,
      key: 'id',
    },
    allowNull: false,
  }
});

// Установление связей
Role.hasMany(User, { foreignKey: 'roleId' });
User.belongsTo(Role, { foreignKey: 'roleId' });

sequelize.sync();
```

2.

Обратите внимание, что связь прописывается в поле roleId у пользователя, а так же после создания модели(со стороны роли многие к одному, со стороны юзера один ко многим).

Шаг 3. Подключаем сессии

С сессиями вы уже работали, они нужны для проверки входа пользователя.

```
const app = express();
app.use(bodyParser.urlencoded({ extended: true }));
app.use(
  session({
    secret: 'secret-key',
    resave: false,
    saveUninitialized: false,
  })
);
```

Шаг 4. Проверка

Далее прописываем саму функцию для проверки входа пользователя

```
app.use(express.static(path.join(__dirname, 'public')));

// Проверка авторизации
function isAuthenticated(req, res, next) {
  if (req.session.user) {
    next();
  } else {
    res.redirect('/login');
  }
}
```

Шаг 5. Проверка роли

```
function hasRole(roleName) {
  return async (req, res, next) => {
    if (req.session.user) {
      const user = await User.findById(req.session.user.id, { include: Role });
      if (user && user.Role.name === roleName) {
        next();
      } else {
        res.status(403).send('Доступ запрещен');
      }
    } else {
      res.redirect('/login');
    }
  };
}
```

Теперь можем создать функцию, для проверки роли

Разбор кода

1. Сигнатура функции

```
function hasRole(roleName) {
```

- Функция принимает параметр `roleName`, который указывает, какую роль должен иметь пользователь для доступа к маршруту.
- Она возвращает другую функцию (асинхронный `middleware`), которая будет вызвана для обработки запроса.

2. Проверка сессии

```
if (req.session.user) {
```

- Проверяется, существует ли в сессии объект пользователя

```
(req.session.user).
```

- Если пользователь не авторизован, он перенаправляется на страницу входа:

```
res.redirect('/login');
```

3. Получение пользователя из базы данных

```
const user = await User.findById(req.session.user.id, { include: Role });
```

- Здесь `findById` используется для получения записи пользователя из базы данных по его `id`, сохранённому в сессии (`req.session.user.id`).
- В запрос включается связанная модель `Role` с помощью `include: Role`. Это позволяет сразу загрузить информацию о роли пользователя.

4. Проверка роли пользователя

```
if (user && user.Role.name === roleName) {
```

- Если пользователь найден (`user` не `null`) и его роль совпадает с требуемой
 - (`user.Role.name === roleName`), вызывается `next()`:
- ```
next();
```

Это передаёт управление следующему `middleware` или обработчику маршрута.

- Если роль пользователя не совпадает с требуемой, возвращается ошибка с кодом **403**:

```
res.status(403).send('Доступ запрещен');
```

### 5. Когда пользователь не авторизован

Если объект пользователя в сессии отсутствует, пользователь перенаправляется на страницу входа:

```
res.redirect('/login');
```

## Шаг 6 прописываем маршруты.

1)

```
app.get('/', (req, res) => {
 if (req.session.user) {
 res.redirect('/profile');
 } else {
 res.redirect('/login');
 }
});
```

2)

```
app.get('/register', async (req, res) => {
 res.sendFile(path.join(__dirname, 'public', 'register.html'));
});
```

3)

```
app.post('/register', async (req, res) => {
 const { login, password, roleId } = req.body;
 try {
 const role = await Role.findPk(roleId);
 if (!role) {
 return res.status(400).send('Роль не найдена');
 }
 (parameter) res: Response<any, Record<string, any>, number>
 res.redirect('/login');
 } catch (error) {
 res.status(400).send('Ошибка регистрации: ' + error.message);
 }
});
```

4)

```
// Роут для страницы авторизации
app.get('/login', (req, res) => {
 res.sendFile(path.join(__dirname, 'public', 'login.html'));
});
```

5)

```
// Роут для обработки авторизации
app.post('/login', async (req, res) => {
 const { login, password } = req.body;
 try {
 const user = await User.findOne({ where: { login, password }, include: Role });
 if (user) {
 req.session.user = { id: user.id, login: user.login, role: user.Role.name };
 res.redirect('/profile');
 } else {
 res.status(401).send('Неверный логин или пароль');
 }
 } catch (error) {
 res.status(500).send('Ошибка сервера: ' + error.message);
 }
});
```

6)

```
app.get('/profile', isAuthenticated, (req, res) => {
 res.send(`Добро пожаловать, ${req.session.user.login}! <a href="/logout">Выйти</a>`);
});
```

7)

```
app.get('/admin', isAuthenticated, hasRole('Администратор'), (req, res) => {
 res.send('Добро пожаловать в админ-панель! Только для администраторов.');
```

Разбор кода

1) Базовый вход на сервер

2) Страница регистрации

- 3) Пост запрос для регистрации
- 4) Страница входа
- 5) Пост запрос для входа
- 6) Страница профиля
- 7) Страница админа (Для определённой роли)

## 5) Пояснение пост запроса для входа

### 1. Получение данных из тела запроса

```
const { login, password } = req.body;
```

- Из тела запроса (`req.body`) извлекаются введенные пользователем `login` и `password`.
- Это происходит благодаря middleware `body-parser` или встроенному в Express `express.urlencoded`.

---

### 2. Поиск пользователя в базе данных

```
const user = await User.findOne({ where: { login, password }, include: Role });
```

- Метод `findOne` ищет первую запись в таблице `User`, которая соответствует логину и паролю, указанным в запросе.
  - `where: { login, password }` — условия поиска.
  - `include: Role` — добавляет информацию о роли пользователя, связанной через модель `Role` (используя связь `User.belongsTo(Role)`).
- Если пользователь найден, объект `user` будет содержать данные пользователя и его роль.

---

### 3. Успешная авторизация

```
if (user) {
 req.session.user = { id: user.id, login: user.login, role: user.Role.name };
};
res.redirect('/profile');
}
```

- Если пользователь найден:
  - В объект сессии (`req.session`) записываются ключевые данные о пользователе:
    - `id` — идентификатор пользователя.
    - `login` — логин пользователя.
    - `role` — название роли (например, "Администратор" или "Пользователь").
  - После этого выполняется перенаправление на страницу профиля (`/profile`).

---

### 4. Неверный логин или пароль

```
res.status(401).send('Неверный логин или пароль');
```

- Если пользователь не найден (объект `user` равен `null`), отправляется HTTP-ответ с кодом **401** (Unauthorized).
- В теле ответа содержится сообщение "Неверный логин или пароль".

## 3) Пояснение пост запроса для регистрации

### 1. Получение данных из тела запроса

```
const { login, password, roleId } = req.body;
```

- Из тела запроса (`req.body`) извлекаются данные, отправленные пользователем через форму регистрации:
  - `login` — логин нового пользователя,
  - `password` — пароль нового пользователя,
  - `roleId` — идентификатор роли, который определяет, какую роль будет иметь новый пользователь.

---

### 2. Проверка существования роли

```
const role = await Role.findByPk(roleId);
if (!role) {
 return res.status(400).send('Роль не найдена');
}
```

- Метод `findByPk` используется для поиска роли в базе данных по её уникальному идентификатору (`roleId`), который был передан в запросе.
- Если роль с таким `roleId` не найдена (`!role`), сервер возвращает ошибку **400** (Bad Request) и сообщение "Роль не найдена".

---

### 3. Создание нового пользователя

```
const user = await User.create({ login, password, roleId });
```

- Если роль существует, создаётся новый пользователь с помощью метода `create` модели `User`. В базу данных записываются:
  - `login` — логин пользователя,
  - `password` — пароль пользователя (его, скорее всего, необходимо хешировать перед сохранением, но это не указано в коде),
  - `roleId` — идентификатор роли, которую получит новый пользователь.
- После выполнения этого метода создаётся новая запись в таблице пользователей.

---

### 4. Перенаправление на страницу входа

```
res.redirect('/login');
```

- После успешной регистрации пользователя происходит перенаправление на страницу входа (`/login`), где пользователь может войти в систему с только что созданными учетными данными.

## 7)Страница админа(пояснения)

### Маршрут

```
app.get('/admin', isAuthenticated, hasRole('Администратор'), (req, res) => {
 res.send('Добро пожаловать в админ-панель! Только для администраторов.');
```

- ```
});
```
- Это HTTP GET запрос на путь `/admin`. Когда пользователь переходит на этот путь, выполняются следующие функции:

2. Проверка аутентификации (*isAuthenticated*)

`isAuthenticated`

- `isAuthenticated` — это middleware-функция, которая проверяет, авторизован ли пользователь. Если пользователь не авторизован (т.е. его нет в `req.session.user`), то его перенаправляют на страницу логина.
- Если пользователь авторизован, обработка запроса продолжается.

3. Проверка роли (*hasRole*)

`hasRole('Администратор')`

- `hasRole('Администратор')` — это middleware-функция, которая проверяет, имеет ли авторизованный пользователь роль "Администратор".
- Для этого функция:
 - Ищет пользователя в базе данных по его `id` (которое хранится в сессии) с включением роли.
 - Если роль пользователя соответствует "Администратор", запрос продолжается.
 - Если роль отличается или пользователь не найден, отправляется ошибка **403 (Forbidden)**, указывающая, что доступ запрещен.

В отчёт необходимо выписать, как работает разделение ролей в вашем проекте, а также описать новые функции.